

Medical Store Stock Management System

Documentation

Project Title: Medical Store Stock Management System

Technology Stack: HTML, CSS, JavaScript, Python Flask, Supabase (PostgreSQL)

Project Type: Web-Based Inventory & Sales Management System

Table of Contents

1. Executive Summary
 2. Project Objectives
 3. System Architecture
 4. Features & Modules
 5. Database Design
 6. API Endpoints
 7. Technical Specifications
 8. Security Implementation
 9. Screenshots & UI Description
 10. Future Enhancements
 11. Conclusion
-

Executive Summary

The Medical Store Stock Management System is a web-based application developed to manage medicines, suppliers, purchases, sales, and stock levels efficiently.

The system allows:

- Managing product inventory
- Tracking supplier details
- Recording purchase orders
- Recording sales transactions
- Automatically updating stock levels
- Displaying analytics on dashboard

The system uses:

- Frontend: HTML, CSS, JavaScript
- Backend: Python Flask (REST API)
- Database: Supabase (PostgreSQL)

This project demonstrates full-stack web development concepts including API development, database integration, and real-time stock management.

2 Project Objectives

Primary Objectives

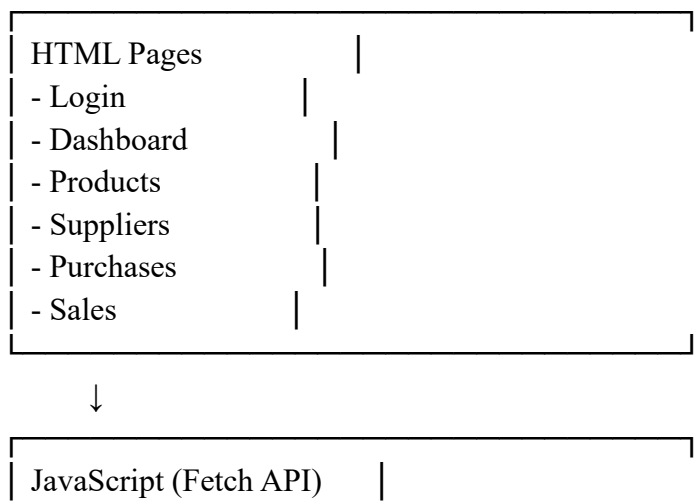
- Develop a user-friendly inventory management system
- Automate stock increase and decrease operations
- Maintain supplier and product database
- Track purchase and sales records
- Provide dashboard analytics

Secondary Objectives

- Implement REST API using Flask
- Integrate Supabase as cloud database
- Provide search and filter functionality
- Maintain clean UI design

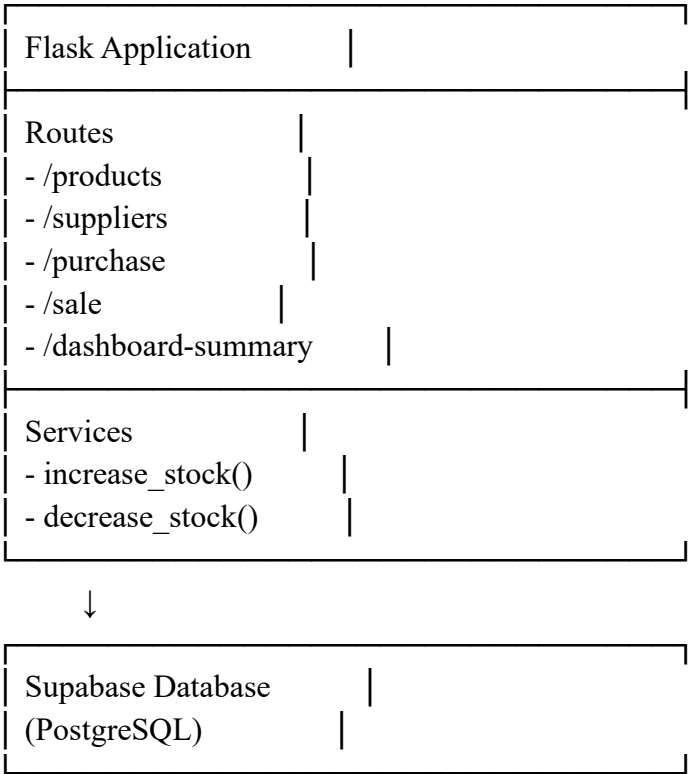
3 System Architecture

Frontend Architecture



- Calls Backend APIs	
- Handles UI updates	

Backend Architecture (Flask)



Features & Modules

4.1 Dashboard Module

- Total Products Count
- Low Stock Count
- Expired Products Count
- Total Inventory Value
- Stock Status Chart
- Recent Purchases
- Top Selling Products
- Stock Alerts
- Expiry Alerts

4.2 Product Inventory

- Add new medicine
- View all products
- Track stock quantity
- Store expiry date
- Search & filter products

4.3 Supplier Management

- Add supplier
- Store contact number
- Store email
- View supplier list
- Search supplier

4.4 Purchase Orders

- Record new purchase
- Increase stock automatically
- View purchase history
- Search purchase records

4.5 Sales Orders

- Record new sale
- Decrease stock automatically
- Prevent sale if stock insufficient
- View sales history
- Search sales records

5 Database Design

Main Tables

1. products

Column	Type
id	UUID (PK)
product_name	text
quantity	integer
price	numeric
expiry_date	date

2. suppliers

Column	Type
id	UUID
supplier_name	text
phone	text
email	text

3. purchases

Column	Type
id	UUID
product_id	UUID (FK)
quantity	integer
total_amount	numeric

4. sales

Column	Type
id	UUID

Column	Type
product_id	UUID (FK)
quantity	integer
total_amount	numeric

6 API Endpoints

Method	Endpoint	Description
GET	/api/products	Get all products
POST	/api/products	Add product
GET	/api/suppliers	Get suppliers
POST	/api/suppliers	Add supplier
GET	/api/purchases	Get purchases
POST	/api/purchase	Create purchase
GET	/api/sales	Get sales
POST	/api/sale	Create sale
GET	/api/dashboard-summary	Dashboard data

7 Technical Specifications

Frontend

- HTML
- CSS
- JavaScript
- Chart.js for graphs

Backend

- Python 3.x
- Flask
- Flask-CORS

Database

- Supabase (PostgreSQL)
 - UUID primary keys
 - Foreign key relationships
-

8 Security Implementation

- UUID-based product identification
 - Stock validation before sale
 - Server-side stock update logic
 - Error handling (400 Bad Request)
 - Separation of frontend and backend
-

9 Screenshots & UI

Dashboard

- Cards displaying totals
- Doughnut stock chart
- Bar chart for inventory value
- Alerts section

Product Page

- Add product form
- Product table
- Search and filter

Sales Page

- Sale form
- Sales history table

Supplier Page

- Add supplier form
- Supplier table

Conclusion

The Medical Store Stock Management System successfully demonstrates:

- Full-stack web development
- REST API implementation
- Cloud database integration
- Real-time stock updates
- Inventory automation logic
- Dashboard analytics

This project reflects strong understanding of:

- Backend API development
- Database management
- Frontend integration
- Business logic implementation